

Adaptive Musical Denoising: A Comparative Evaluation of Hybrid, Fine-Tuned and retrained DeepFilterNet Architectures

Yunze Liu, Ruizhe Zhang, Xiankai Guo

Abstract—Most current noise cancellation systems are typically optimized for speech and often suppress musical instruments as background noise. This project investigates methods for distinguishing unwanted environmental noise from desirable musical content during denoising. We initially explored a hybrid architecture that combined DeepFilterNet with the lightweight classifier YAMNet to adapt suppression strength without retraining. However, latency constraints prevented real-time operation, and offline evaluation showed that this hybrid approach provided limited acoustic quality and poor music preservation. Further experiments demonstrated that directly fine-tuning DeepFilterNet yielded better overall performance. Our best fine-tuned model, using a 0.5 music ratio and 64 ERB, achieved a Fréchet Audio Distance of 1.28 and a Perceptual Evaluation of Speech Quality score of 2.0, compared with 5.6968 and 1.12 for the hybrid model. These results show that fine-tuning the denoising model is more effective than external adaptive control for preserving musical fidelity while reducing unwanted noise.

I. INTRODUCTION

Clear audio is incredibly important for communication, whether it is a Zoom call, a gaming stream, or a remote music lesson. While humans are very good at focusing on a voice while still hearing a background instrument, computers still struggle with this complex acoustic task. Although Artificial Intelligence has gotten much better at removing noise, it often makes mistakes when the audio environment gets complicated, like in a music room. The main problem is that standard noise cancellation models are too aggressive. For example, in preliminary testing with the DeepFilterNet model, we found that setting the attenuation limit (noise reduction strength) to 100 dB creates a very quiet background for speech, but it completely removes the sound of musical instruments. On the other hand, manually lowering the strength to let the instruments through also lets in a lot of static and room noise, and it is entirely impractical to manually adjust this setting every second during a live performance. Initially, we considered retraining DeepFilterNet to recognize music but assumed the cost would be substantial because the training process requires more time and computational power than we had available. To circumvent this, we proposed a faster, hybrid model utilizing a pre-trained DeepFilterNet as the main engine for suppression and integrating YAMNet to act as a real-time control component. The intent was for YAMNet to detect musical instruments and automatically reduce the noise reduction strength of DeepFilterNet, increasing it again when only noise was present. However, the practical implementation of this system yielded two critical insights that

shifted the direction of our research. First, achieving real-time integration proved unfeasible; processing audio blocks using PyTorch and DeepFilterNet significantly exceeded our 40ms target latency. Consequently, our evaluation methodology transitioned to offline processing to accurately measure system efficacy without hardware-induced lagging. Second, comparative testing revealed that our initial assumption regarding retraining was flawed. While the hybrid approach functioned as an adaptive switch, fine-tuning and fully retraining the DeepFilterNet model generated an inherently superior model capable of balancing speech enhancement and musical fidelity more effectively than the hybrid logic.

This paper expands upon these findings and contributes the following to the field of audio processing:

- 1) **Introduction of an Adaptive Hybrid Pipeline:** We design a system that connects a pre-trained audio classifier (YAMNet) with a deep filtering enhancer (DeepFilterNet) to control noise suppression heuristically.
- 2) **Fine-Tuning and Retraining Methodologies:** We develop and apply techniques to fine-tune and fully retrain the DeepFilterNet model, teaching it to naturally preserve musical frequencies without requiring an external classifier.
- 3) **Methodological Shift for Metric Extraction:** We transition from real-time streaming assumptions to an offline processing framework that enables accurate evaluation with full-reference acoustic metrics.
- 4) **Comparative Performance Benchmarking:** We conduct a comprehensive offline evaluation comparing the initial hybrid approach against the fine-tuned and retrained DeepFilterNet models, demonstrating that direct fine-tuning achieves a better balance of speech clarity and musical preservation.

II. RELATED WORKS

A. Real-time Enhancement Backbone: DeepFilterNet

In a hybrid instrument-aware noise suppression system, the real-time enhancement backbone serves as a stable, low-latency default denoiser. Instead of retraining the enhancer for mixed speech-music conditions, the hybrid design keeps the backbone fixed and uses a separate controller to modulate suppression strength when instruments are present. In this case, we consider DeepFilterNet. DeepFilterNet is a low-complexity speech enhancement framework designed for full-band audio and real-time deployment [1]. Instead of using

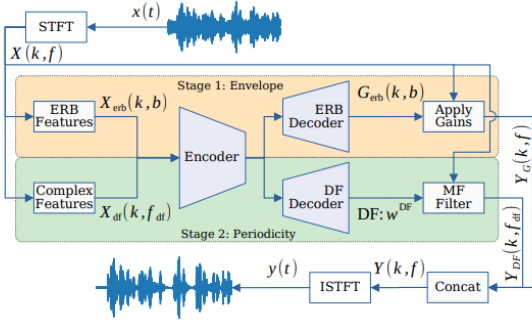


Fig. 1. DeepFilterNet architecture overview. [9]

only pointwise-frequency masks, DeepFilterNet incorporates “deep filtering” by estimating complex-valued filters that can exploit short temporal contexts to help improve signal quality while keeping computation and model size practical. For an instrument-aware system, DeepFilterNet can be treated as a fixed enhancer, such that the main research effort shifts from rebuilding a new backbone to deciding how strongly the backbone should suppress noise when instruments are present. In addition, because DeepFilterNet models phase/temporal structure, overly strong suppression may still produce perceptually salient artifacts or cause destructive results on structured signals like music or instruments, which motivates adaptive suppression control rather than static settings [1].

B. Audio and instrument classification

Dynamic audio classification requires efficient models like YAMNet, which is built on the AudioSet dataset and the MobileNetV1 architecture [2]. Rather than processing raw 1D audio waveforms, YAMNet converts overlapping audio frames into spectrograms. This creates a 2D matrix representing time and frequency, allowing the network to process the acoustic signal using computer vision techniques. MobileNetV1 achieves its high efficiency through depthwise separable convolutions. Standard convolutional layers perform massive matrix multiplications across all channels simultaneously. In contrast, MobileNetV1 applies a spatial filter to each channel individually, followed by a 1x1 pointwise convolution to combine the outputs. This drastically reduces computational operations, enabling YAMNet to classify hundreds of audio events, including musical instruments with very low latency and minimal processing power [2]. Additionally, YAMNet serves as an effective feature extractor within larger hybrid networks. For example, a recent study combined YAMNet with a Bidirectional Long Short-Term Memory (LSTM) network to classify overlapping urban sounds [6]. Instead of outputting final class probabilities, YAMNet generates a 1024-dimensional embedding vector containing the critical acoustic characteristics of the signal [6]. The LSTM then uses these pre-calculated embeddings to track temporal changes and make a final classification, improving accuracy without the need to train a new feature extraction model from scratch [6].

C. Evaluation Methods: DNS Challenge

In enhancement tasks, evaluation can be misleading if it only focuses on “denoise strength” or purely synthetic mixtures. The instrument-aware goal is multi-objective, which is speech quality, and musical fidelity. The INTERSPEECH 2020 Deep Noise Suppression (DNS) Challenge pushed the field towards realistic evaluation and perceptual quality [3]. A key message is that performance on synthetic mixtures may not transfer to real recordings, and objective metrics alone can be unreliable. DNS emphasized evaluation with real recordings and scalable subjective testing. For instrument-aware denoising, this is an important constraint in which improvements should be judged by whether users perceive better quality among speech clarity and music fidelity, not only by standard denoising metrics [3].

D. Datasets for Instrument Awareness

A hybrid suppression controller needs a context signal, such as instrument activity. Two datasets provide complementary support: one gives broad coverage for general audio tagging, and the other supports music-domain calibration:

1) *AudioSet*: AudioSet provides a large-scale ontology of audio events with human-labeled 10 seconds clips from YouTube [2]. It covers many musical and non-musical classes, so it is a strong foundation for training or using pretrained audio tagging models that output music/instrument probabilities [2]. In a hybrid system, such probabilities can directly drive the suppression controller. A limitation is that the labeling is weak due to its nature of clip-level presence/absence and its broad domain. In this way, it can lead to uncertain predictions under challenging acoustics [2].

2) *Isolated Instrument-domain: Music Instrument Sounds for Classification*: Music instrument sounds for classification provides short, isolated instrument recordings that are useful for teaching a model what musical content should be preserved rather than attenuated. The dataset contains a large scale of audio files, with 3-second clips from 28 instrument categories [16], which makes it practical for instrument-domain learning. However, a limitation is that the clips are mostly isolated and short, so they do not fully demonstrate polyphonic mixtures or real performance, and the class is also distributed unevenly across instruments.

3) *OpenMIC*: OpenMIC-2018 focuses on multi-instrument recognition in polyphonic music [4]. Compared to general event datasets, it better reflects real musical mixtures and is suitable for validating and calibrating instrument detection behavior. This matters because detector errors translate into audible outcomes, that is, false positives may weaken suppression and leak noise, while false negatives may trigger over-suppression that damages instruments. OpenMIC therefore supports practical tuning steps (threshold selection, confidence calibration, light domain adaption) to make the context signal more reliable for control [4].

4) *Speech-domain: Noisy speech database for training speech enhancement algorithms and TTS models*: The dataset provides both clean and noisy speech (parallel) at 48kHz, with

training material built from 29-speaker and 56-speaker subsets and matched test data [17]. This makes it suitable for our work, because the clean/noisy pairing gives a direct guidance for learning speech preservation.

5) *Noise-Domain: UrbanSound8K*: UrbanSound8K provides 8,732 labeled urban sounds of up to 4 seconds from 10 classes. In our work, we used only a random subset of 2,000 clips [18]. In the setting, these recordings support the noise-domain of the mixtures by introducing urban sounds that would challenge the music-domain and music-speech-domain mixtures.

6) *Supplementary Noise-domain: Audio Noise Dataset*: Audio Noise Dataset contributes 10 categories of real-world background noise collected in everyday settings, including crowded places, festivals, workplaces, rain, and more [19]. It is useful as a supplementary noise source because it expands the variety of the noise patterns during training and helps validate robustness. The limitation is that the size of the dataset is small and the audios are recorded under very limited conditions.

E. Control Strategy: MetricGAN

For control strategy, a common baseline is heuristic switching, that is, if music is detected, reduce suppression. However, this can be unstable when detection is uncertain, and it may not align with perceptual quality. There exist some methods, for example, MetricGAN, highlights commonly used training losses may not match perceptual evaluation metrics [5]. It proposes learning a differentiable surrogate, so optimization directly targets metric improvement. In an instrument-aware controller setting, MetricGAN can be used as a guiding principle: rather than rely only on hand-designed rules, we can tune a small set of controller parameters against perceptually motivated evaluation on a development set. Learning a metric surrogate and training the controller end-to-end is a natural extension beyond a short project cycle [5].

F. Hybrid Approaches to Audio Denoising

Standard deep learning models for noise suppression require high computational power, and they frequently remove non-speech audio, such as music, because their training data only defines human speech as the target signal. To solve this, researchers use hybrid models that combine different signal processing algorithms to separate the calculations. One proven method combines traditional mathematical digital signal processing (DSP) with a Recurrent Neural Network (RNN). In this structure, the system groups the audio frequencies into specific bands that match human hearing limits. The RNN does not calculate the output for the entire audio wave. Instead, the network only calculates the required volume reduction limit for each specific frequency band. Concurrently, a traditional DSP pitch filter operates on the signal to reduce the noise located between the fundamental frequency harmonics of the audio. This limits the number of variables the neural network must learn. [9] Other hybrid approaches use mathematical signal transformations to prepare the data before it enters the neural network. Research shows that applying a Discrete

Wavelet Transform (DWT) as a pre-processing step improves the accuracy of Convolutional Neural Networks (CNNs) [8]. Standard transformations apply a fixed time window to analyze all frequencies. The DWT applies to variable window sizes. It uses short-term windows to analyze high frequencies, which captures rapid changes in the audio. It uses long time windows to analyze low frequencies, which captures highly accurate pitch values. When the CNN processes these separated DWT coefficients instead of the standard audio matrix, the network layers can calculate the numerical differences between speech patterns and noise patterns with higher accuracy [8]. Systems can also maximize computational efficiency by routing different frequency ranges to different types of algorithms. Speech energy is concentrated in lower frequencies, and environmental reverberation also causes the most data loss in these low frequencies. The MDPI framework divides the audio by frequency limit. For the high frequencies, the system applies an Ideal Ratio Mask, which is a simple multiplication function that reduces the amplitude of broadband noise. For the low frequencies, the system applies to a complex-valued deep neural network. This deep network requires many calculations because it must output corrections for both the amplitude and the phase angle of the audio wave. By restricting this complex network to only the low frequencies, the system reduces the total processing time. [7] Hybridization is also applied directly to the internal architecture of neural networks to process multiple data dimensions simultaneously. Models such as DeepFilterNet combine CNNs and RNNs to achieve high accuracy. The CNN layers extract local frequency features and spatial features from the 2D audio matrix. The RNN layers use Gated Recurrent Units to calculate how these features change across sequential time steps. To further divide the processing, the architecture splits the input data into two streams. The first stream analyzes the full frequency band to calculate the global pitch and speech characteristics. The second stream analyzes narrow sub-bands independently to calculate the localized noise properties for specific frequency ranges. The system then integrates the outputs of both streams to generate the final processed audio. [10]

G. Continuous and Temporally Stable Control

Recent speech enhancement research increasingly recognizes that perceptual quality depends not only on the amount of noise suppression, but also on how suppression strength is represented and controlled over time, particularly in real-time systems. Discrete or abruptly switching suppression strategies often introduce temporal instability or unnecessary signal distortion under non-stationary noise, motivating the use of continuous-valued suppression representations with stable temporal behavior. A representative neural approach is the convolutional recurrent neural network (CRN) proposed by Zhao et al., which estimates continuous time-frequency masks and models temporal dependencies through recurrent layers [11]. Their results demonstrate improved robustness in non-stationary noise conditions, aligning with the idea of continuous regression-based suppression control. Real-time

constraints further emphasize the importance of stable suppression trajectories. Westhausen and Meyer introduce the Dual-Signal Transformation LSTM (DTLN), a low-latency enhancement framework designed for streaming applications [12]. Their work shows that perceptual quality in real-time systems is strongly influenced by the temporal smoothness of enhancement outputs. The preference for continuous suppression representations is also grounded in classical enhancement theory. Ephraim and Malah formulate speech enhancement as a continuous gain estimation problem through the MMSE short-time spectral amplitude estimator [13]. While not a neural or real-time system, this foundational work provides theoretical justification for treating suppression strength as a continuous quantity rather than a binary on–off operation, a principle that remains influential in modern enhancement designs. Beyond representation, the aggressiveness of suppression must be carefully controlled to balance noise reduction and signal fidelity. Xia et al. study this trade-off by introducing weighted speech distortion losses for real-time neural enhancement systems, enabling systematic control over suppression aggressiveness during training [14]. Finally, Takeuchi et al. propose an Equilibrated Recurrent Neural Network (ERNN) for real-time speech enhancement that emphasizes stable output behavior under strict latency constraints [15]. Their causal design demonstrates that lightweight recurrent architectures can maintain temporal consistency in streaming scenarios. Taken together, these works suggest that continuous and temporally stable suppression control is not merely a modeling choice, but a necessary design principle for real-time systems that must balance noise reduction and signal fidelity.

III. METHODS

This section details the progression of architectures and training paradigms developed to address the multi-objective problem of preserving both speech and musical fidelity in noisy environments.

A. Hybrid Control via YAMNet and DeepFilterNet

Method 1 used two existing models together: YAMNet and DeepFilterNet. Instead of changing how DeepFilterNet was trained, we changed how strongly it removed noise based on the type of sound in the audio. DeepFilterNet has a setting called `atten_lim_db` [1], which controls the maximum amount of noise reduction. A high value, such as 100 dB, works well for removing normal background noise and keeping speech clear. However, it can also remove parts of music, which we wanted to keep. To solve this, we built a control model that chooses a better attenuation limit based on the sound environment.

1) *Extracting sound features:* To understand the audio environment, we first resampled the input audio to 16 kHz and passed it through YAMNet. YAMNet is a pre-trained model based on MobileNetV1 that can recognize many different types of sounds. It gives probability scores for 521 sound classes for each audio frame.[2] We averaged these scores over time to

create one 521-dimensional feature vector for each audio clip. This vector describes the overall sound scene.

2) *Creating training targets and scoring each setting:*

To train the control model, we needed examples that linked each audio mixture to its best attenuation limit. We created a synthetic dataset by mixing separate speech, music, and noise clips at fixed volume levels of 1.0, 0.6, and 0.8. The target clean signal was defined as speech plus music, so only the background noise should be removed.

Next, we found the best attenuation limit for each mixture using a two-step grid search. We first tested values from 0 dB to 100 dB in steps of 10 dB. Then we tested more closely around the best result from that first search, within plus or minus 9 dB.

To choose the best value, we used a custom score that balanced two goals: keeping the signal close to the target and preserving intelligibility. The score combined SI-SDR and STOI with equal weight, defined as

$$C = w_{\text{SDR}} \cdot \text{SDR}_{\text{norm}} + w_{\text{STOI}} \cdot \text{STOI} \quad (1)$$

where the weights were evenly distributed ($w_{\text{SDR}} = 0.5$ and $w_{\text{STOI}} = 0.5$). Since SI-SDR can vary widely and could dominate the final score, we normalized it to a range from 0 to 1 using an assumed practical range of -10 dB to 20 dB:

$$\text{SDR}_{\text{norm}} = \max \left(0, \min \left(1, \frac{\text{SI-SDR} + 10}{30} \right) \right) \quad (2)$$

3) *Training the control model:* After building the dataset, we trained a model to predict the best attenuation limit from the 521 YAMNet features. We used an XGBoost regressor because it works well on high-dimensional tabular data and is less likely to overfit. The model was trained with mean squared error loss. To allow smooth and detailed control, we used 5000 estimators and a small learning rate of 0.01, with early stopping to prevent overfitting.

The final XGBoost model served as a smart controller. It used YAMNet’s sound-scene information to automatically choose how strongly DeepFilterNet should reduce noise.

B. Teacher-Student Architecture

Method 2 took a different approach. Instead of only adjusting DeepFilterNet’s settings, we trained a new and smaller Convolutional Neural Network to remove noise on its own. This model worked in a teacher-student setup. DeepFilterNet acted as the teacher and produced high-quality training targets, while the smaller CNN acted as the student and learned to copy that behavior. The goal was to remove background noise while keeping both speech and music unchanged. However, the results of music overall FAD of 17.6864 show that it did not learn anything useful from the dataset; therefore, we do not include it in the comparison with the other models.

1) *Creating the training data:* To train the student model, we built a custom dataset from separate speech, music, and noise clips. Each clip was first converted to the correct sample rate, changed to mono if needed, and padded or trimmed so

all samples had the same length. In the data generation code, each training example was 3 seconds long.

Next, we had our teacher model (DeepFilterNet) clean up this noisy speech track. To make things easier for our lightweight student model to process, we lowered the audio quality by downsampling it from 48 kHz to 16 kHz. The final data pairs given to the student model looked like this:

- The Input: Noisy speech + unaltered music.
- The Target: teacher-cleaned speech with unaltered music.

By doing this, the model learns that noise is the only thing it needs to remove, and that musical frequencies are important and should be left alone.

2) *How the model processes sound:* The student model worked in the frequency domain instead of directly on the raw waveform. It first converted the input audio into a Short-Time Fourier Transform (STFT), which represents the signal as time-frequency information. From this, the model separated the magnitude and phase. The magnitude shows how strong each frequency is, while the phase keeps timing information. Only the magnitude was passed into the neural network. The network itself was small and used three 1D convolutional layers with batch normalization and PReLU activation. The middle layer used dilation so the model could look at a wider time context without becoming too large. The final layer used a sigmoid activation so that the output became a mask with values between 0 and 1. This mask acted like a filter. It told the system how much of each time-frequency component should be kept and how much should be reduced. In this way, the model learned to suppress parts of the signal that looked like noise while preserving useful speech and music content.

3) *Rebuilding the clean audio:* After predicting the mask, the model multiplied it with the original noisy magnitude. This reduced unwanted noise in the frequency representation. The original phase was then added back, and the signal was converted to waveform audio using the inverse STFT. This produced the final denoised output.

C. Fine tuning DFN

Our third method keeps the DeepFilterNet structure unchanged and adapts an existing pretrained DeepFilterNet3 model_120 checkpoint to our target scenarios [9]. The motivation is simple, the released pretrained DeepFilterNet3 model_120 already offers a strong real-time speech-enhancement prior and a stable setup, so fine-tuning provides a quick way to test whether the model can be redirect toward our goal without retraining from scratch. In our work, this is the lowest-risk path: preserve the official architecture and the strong baseline on speech, only update the weights enough to expose the model to our mixtures of speech, music, and environmental noise.

The fine-tuning setup strictly follows the official DeepFilterNet3 configuration. In the implementation, the model is initialized from the official checkpoint directory rather than from random weights [9]. In this way, the structure and the performance of the original system are not altered.

Instead of exposing raw waveforms directly to the training, we build manifests containing aligned clean and noisy examples and then cache DeepFilterNet-ready features. For speech-domain, the supervision comes from VoiceBank paired clean and noisy audios; for music-domain, clean music clips are mixed with sampled environmental noise at randomized SNRs. In the fine-tuning pipeline, these examples are assembled into a 70% music ratio manifest with precomputed `noisy_spec`, `erb_feat`, `spec_feat`, and `clean_spec`, which reduces processing time each epoch and makes the training loop more reproducible [9].

A key decision is the training objective. In the fine-tuning version, we use a conservative loss that stays close to the released model_120 [9]. Rather than activating multiple terms, the method applies a multi-resolution spectrogram loss after reconstructing both prediction and target back into the time domain with ISTFT [20]. Denoted as:

$$L_{FT} = L_{MRSpec}(\text{ISTFT}(S_{\text{predicted}}), \text{ISTFT}(S_{\text{clean target}})) \quad (3)$$

Optimization is also conservative. Fine-tuning uses AdamW with a small learning rate of $1e^{-5}$, moderate weight decay, and gradient clipping [9]. The batch size is also kept relatively small compared with the later retraining experiments. In practice, it converged quickly and stably. Fine-tuning runs on 20k-scale manifests reached their best validation values within roughly 20 to 30 epochs.

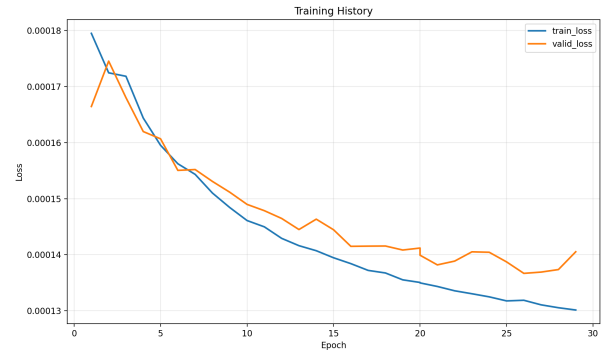


Fig. 2. Fine tuning trending.

From the perspective of our project, the main strength of fine-tuning is efficiency. It reuses the official DeepFilterNet3 prior, preserves the real-time architecture, and requires fewer epochs to reach a usable checkpoint. It is therefore a very attractive starting point. However, conservatism makes it stable but also limits how far the model can move. The official checkpoint is initially speech-oriented, and in the fine-tuning route we also keep the original `nb_erb` and leave the mask loss disabled [9]. As a result, adaptation ability is limited when the target domain includes not only speech but also instrumental mixtures and speech_music cases.

This observation is what motivated our fourth method. Once fine-tuning had shown that the official DeepFilterNet3 release

could be adapted stably but not completely reshaped, we moved to another strategy in which the model is retrained directly on our own mixed-domain data.

D. Retraining DFN

Our fourth method retraining a DeepFilterNet3 model from scratch on our dataset and loss configuration [9]. The motivation is to remove the strong dependence on the official speech-enhancement prior and let the model learn directly from the data distribution we care about: paired noisy speech, musical instrument mixed with environmental noise, and evaluation scenarios that include both speech and instrument. Compared with Method 3, this approach is more computationally heavy and more sensitive to hyperparameters, but it gives us much more freedom to shape the representation rather than only adapting the previous baseline. Although Method 4 keeps DeepFilterNet3 as the backbone, it is not an exact copy of the official configuration. The most important architectural change is the increase from $\text{nb_erb} = 32$ in the official release to $\text{nb_erb} = 64$ in our retraining configuration. This doubles the number of perceptual envelope bands while keeping the rest unchanged [9]. In this case, we preserve the same latency and the same multi-frame filtering structure, but ask the model to represent the spectral envelope at a finer ERB resolution. For a speech-only denoiser, the 32-band setup is already highly effective. However, for our project, music preservation matters, so a finer ERB resolution gives the model more opportunities to differentiate instrumental timbres and musical energy from background noise. We also keep $\text{mask_pf} = \text{False}$ and $\text{nb_df} = 96$, because our own attempts with the post-filter enabled did not produce the listening behavior we wanted and instead led to heavy artifacts after repeated testing.

The retraining pipeline still uses three major sources: paired VoiceBank speech, a large instrument-oriented music corpus, and an environmental noise pool similar to the ones discussed in Method 3. All audio is converted into 48 kHz mono, segmented into 3-second excerpts, and then divided into train, validation, and test splits. Music examples are synthesized by mixing clean music with environmental noise at randomized SNRs from -5 to 20 dB, while speech supervision is preserved through the original clean and noisy VoiceBank pairing.

A useful fact about this retraining dataset is that it is balanced with a music ratio of 50%. Our manifest design aims to expose the model to different use cases while preserving the music-aware denoising problem that motivated the project.

The retraining objective is also more expressive than the fine-tuning objective. Instead of relying only on a multi-resolution spectrogram loss, the current scratch-training implementation uses DeepFilterNet’s built-in Loss wrapper and activates selected terms [9]. The resulting training objective can be summarized as:

$$\mathcal{L}_{\text{tot}} = \lambda_{\text{mask}}\mathcal{L}_{\text{mask}} + \lambda_{\text{mrs}}\mathcal{L}_{\text{MRSspec}} + \lambda_{\text{snr}}\mathcal{L}_{\text{SNR}} \quad (4)$$

where in the configuration, $\lambda_{\text{mask}} = 0.1$, $\lambda_{\text{mrs}} = 500$, $\lambda_{\text{snr}} = 10\text{e-}3$. Other losses, including the explicit spectral loss, DF-alpha loss, SDR loss, and ASR loss, remain disabled [9]. The

maskloss setting uses $\text{mask} = \text{spec}$, meaning that the mask term is not a simple ideal-amplitude. Instead, it combines the predicted masking behavior to the clean target structure. This is useful in our setting because it ensures the prediction to be both clean in content and high in quality.

Optimization in Method 4 is intentionally more aggressive than in Method 3 because there is no pretrained checkpoint to protect. We then optimize with AdamW at a learning rate of $5\text{e-}4$, batch size 64, small weight decay of $1\text{e-}12$, and gradient clipping. The current 64-ERB retraining reduced validation loss from approximately 1.94 at epoch 1 to approximately 1.37 at epoch 34, after which training stopped. The stop could be triggered by either early stopping or manually. One thing we observed is that while the loss continued to go downward beyond around Epoch 40, we observed an increase in audible artifacts and echo, causing a drop in both perceptual quality and metrics in most testing.

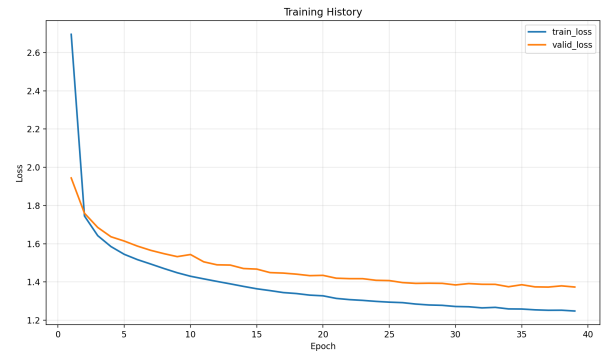


Fig. 3. Retraining trending.

Retraining from scratch changes the role of DeepFilterNet in the project. Under Method 3, DeepFilterNet somehow remains a speech enhancer that we adapt. Under Method 4, it becomes the backbone of a new domain-specific denoiser where the representation is learned for our data mixture, such that it can better learn what should be preserved and what should be attenuated.

Method 4 is interesting. Our experiments find that this training path is sensitive to ERB resolution, DF resolution, dataset balance, and loss weighting. For example, earlier retraining runs with 48 ERB bands and with 64 ERB bands but no mask loss did not always produce the same perceptual behavior, and increasing the dataset size from roughly 40k to 50k examples did not automatically improve listening quality.

IV. REAL TIME DENOISING ATTEMPTS AND LIMITATIONS

A. Keeping the stream state and limiting spectral changes

In real-time streaming, the model processes audio one block at a time. A simple but important issue is how the recurrent state is handled. If the GRU state is reset at every block, the output can change too much from one block to the next, which may cause periodic pumping artifacts. To reduce this problem, we kept the stream state across consecutive blocks and only

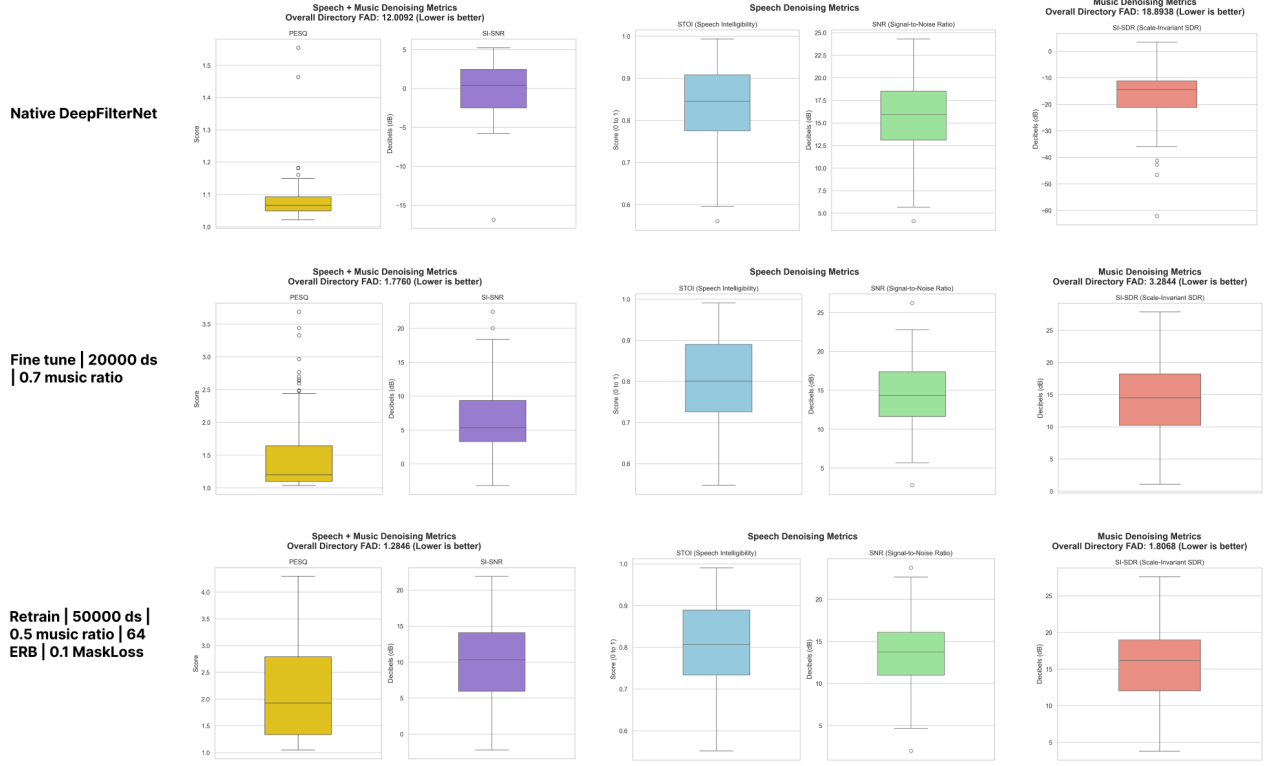


Fig. 4. Box plots comparing the performance of the evaluated models across objective metrics.

reset it at the beginning of a recording session. We also tested a spectral attenuation limit before iSTFT. Instead of using the enhanced spectrum alone, we mixed a small part of the input spectrum back into the output. This made the enhancement less aggressive and helped reduce unstable changes between neighboring blocks.

$$\hat{S}' = \lambda S + (1 - \lambda) \hat{S} \quad (5)$$

where S is the input spectrum, $\hat{S}$ is the enhanced spectrum, and λ controls how much of the input spectrum is mixed back. In practice, these two changes made the real-time output more stable, but they did not fully remove audible artifacts. They were useful as simple stabilization steps, but by themselves they were not enough to guarantee consistently clean real-time denoising.

B. Smoothing the join between blocks

Even after keeping the stream state, small artifacts could still appear where two output blocks were joined together. To make these transitions smoother, we added a short crossfade between neighboring blocks. In this step, the end of the previous block and the beginning of the next block were blended over a small overlap region instead of being connected directly.

$$y[n] = (1 - \alpha[n])y_{\text{prev}}[n] + \alpha[n]y_{\text{next}}[n] \quad (6)$$

where $y_{\text{prev}}[n]$ is the tail of the previous block, $y_{\text{next}}[n]$ is the start of the next block, and $\alpha[n]$ increases from 0 to 1 across the overlap region.

To keep the processing consistent with the streaming setup, the overlap length was limited and aligned to the hop size. This made the block boundaries sound a little smoother and reduced some local discontinuities.

In our tests, this helped at some boundaries, but the overall improvement was limited. It could reduce small joining artifacts, but it did not solve the main real-time instability by itself.

C. Dry mix from RMS and ZCR

For each audio block, we adjusted the dry mix using simple RMS (Root Mean Square) and ZCR (Zero Crossing Rate) rules. The goal was to keep denoising strong when speech, music, or other clear foreground content was present, but allow a small amount of the raw microphone signal to return during quiet gaps so that silence sounded more natural.

$$y[n] = (1 - m)z[n] + mx[n], \quad m \in [0, m_{\text{max}}] \quad (7)$$

where $z[n]$ is the enhanced output, $x[n]$ is the original microphone signal, and m is the dry-mix value.

We did not change the dry mix value abruptly. Instead, we smoothed it with a fast attack and a slow release. This means the system could return to strong denoising quickly when

TABLE I

QUANTITATIVE COMPARISON OF THE EVALUATED MODELS USING SPEECH AND MUSIC DENOISING METRICS. FOR FAD, LOWER IS BETTER. FOR PESQ, STOI, SNR, SI-SNR, AND SI-SDR, HIGHER IS BETTER.

Model / Configuration	Speech + Music FAD (Overall)	PESQ (Median)	SI-SNR (Median)	STOI (Median)	SNR (Median)	Music FAD (Overall)	SI-SDR (Median)
Native DeepFilterNet	12.0092	1.0818	-0.0507	0.8384	15.7223	18.8937	-17.1756
Fine tune 20000 ds 0.7 music ratio	1.7760	1.4955	6.5954	0.8016	14.3294	3.2844	14.0075
Retrain 50000 ds 0.5 music ratio 64 ERB 0.1 MaskLoss	1.2846	2.1351	10.3427	0.8056	13.5278	1.8068	15.7855
YamNet controls the slider in DFN	5.6968	1.1576	-0.9699	0.5393	2.8230	10.8673	-0.9638

content came back, but only increased the dry mix slowly in quieter parts.

In practice, this method did not work equally well in every condition. When denoising was too strong, the control could still change too quickly and cause pumping or stuck-like artifacts. When we used smoother control, these artifacts were reduced, but more residual noise remained. This shows a clear trade-off: in the current real-time setup, we could not achieve both very strong denoising and very stable output at the same time.

V. COMPARISON

Native DeepFilterNet is good at protecting speech intelligibility (STOI 0.8384), but it is terrible at preserving music. Its Music FAD is 18.8937 and SI-SDR is -17.1756 dB, which suggests it removes or damages the music too aggressively.

YAMNet controlling the slider is better than native DeepFilterNet for music preservation, but still not strong overall. It has weak speech-related scores like STOI 0.5393 and SNR 2.8230 dB, so the adaptive control idea did not work as well as hoped.

Fine-tuning DeepFilterNet is a big improvement. It gets much lower Speech+Music FAD (1.7760) and Music FAD (3.2844), while also giving much stronger denoising scores like SI-SNR 6.5954 dB and SI-SDR 14.0075 dB. This means it keeps a much better balance between cleaning noise and keeping music.

Retraining DeepFilterNet from scratch is the best overall. It achieves the best Speech+Music FAD (1.2846), PESQ (2.1351), SI-SNR (10.3427 dB), Music FAD (1.8068), and SI-SDR (15.7855 dB). This model therefore provides the strongest overall quality and the best music preservation.

The retrained model is not the best on STOI or SNR. Native DeepFilterNet has slightly higher STOI and SNR, but it achieves that by being too aggressive and destroying musical content. So the retrained model is still the better choice because your project goal is to preserve both speech and music, not speech alone.

In the spectrogram comparison Fig. 5, the diagonal streaks correspond to background noise. Native DeepFilterNet removes these streaks aggressively, but also introduces large blank regions, indicating over-suppression and loss of useful speech/music content. The fine-tuned and retrained models preserve more spectral structure while still reducing the diagonal noise. Among them, the retrained model provides the best

balance, showing effective noise removal with less damage to the underlying signal.

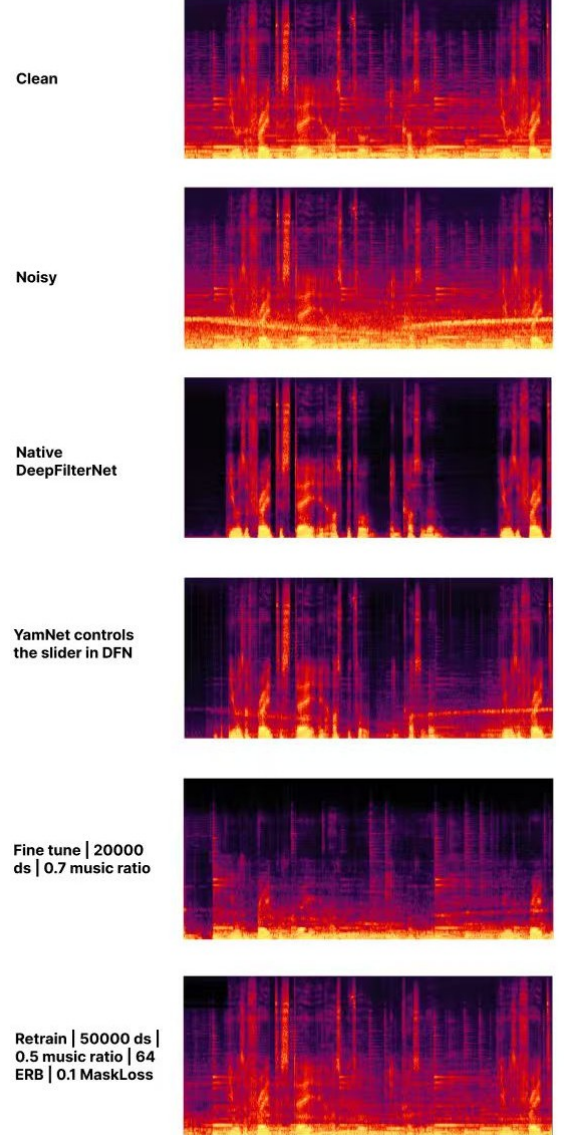


Fig. 5. Retraining trending.

VI. CONCLUSION

This project showed that removing noise from audio while keeping music clear is harder than normal speech denoising.

Our first idea, a hybrid system that used YAMNet to control DeepFilterNet, did not work as well as we hoped. It had real-time latency problems and did not preserve music strongly enough. In comparison, directly adapting DeepFilterNet gave much better results. Fine-tuning improved both denoising quality and music preservation, and retraining the model from scratch gave the best overall performance. Overall, our results show that teaching the denoising model to handle music directly is more effective than using an external controller. Future work can focus on making the system more stable for real-time use and testing it on more real-world recordings.

REFERENCES

- [1] H. Schröter, A. N. Escalante-B., T. Rosenkranz, and A. Maier, “DeepFilterNet: A low complexity speech enhancement framework for full-band audio based on deep filtering,” *arXiv preprint arXiv:2110.05588*, 2021, doi: 10.48550/arXiv.2110.05588.
- [2] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, “Audio Set: An ontology and human-labeled dataset for audio events,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2017, pp. 776–780, doi: 10.1109/ICASSP.2017.7952261.
- [3] C. K. A. Reddy, V. Gopal, R. Cutler, E. Beyrami, R. Cheng, H. Dubey, S. Matushevych, R. Aichner, A. Aazami, S. Braun, P. Rana, S. Srinivasan, and J. Gehrke, “The INTERSPEECH 2020 Deep Noise Suppression Challenge: Datasets, subjective testing framework, and challenge results,” in *Proc. Interspeech*, 2020, pp. 2492–2496, doi: 10.21437/Interspeech.2020-3038.
- [4] E. J. Humphrey, S. Durand, and B. McFee, “OpenMIC-2018: An open dataset for multiple instrument recognition,” in *Proc. 19th Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, Paris, France, 2018, pp. 438–444.
- [5] S.-W. Fu, C.-F. Liao, Y. Tsao, and S.-D. Lin, “MetricGAN: Generative adversarial networks based black-box metric scores optimization for speech enhancement,” in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, vol. 97, 2019, pp. 2031–2041.
- [6] T. Wu, “A comprehensive approach to urban sound detection with YAMNet and bi-directional LSTM,” in *Proc. 2nd Int. Conf. Data Analysis and Machine Learning*, vol. 1, 2024, pp. 23–29, doi: 10.5220/0013486700004619.
- [7] T. Rosenbaum, E. Winebrand, O. Cohen, and I. Cohen, “Deep-learning framework for efficient real-time speech enhancement and dereverberation,” *Sensors*, vol. 25, no. 3, p. 630, 2025, doi: 10.3390/s25030630.
- [8] P. Cherukuru and M. B. Mustafa, “CNN-based noise reduction for multi-channel speech enhancement system with discrete wavelet transform preprocessing,” *PeerJ Computer Science*, vol. 10, p. e1901, 2024, doi: 10.7717/peerj-cs.1901.
- [9] H. Schröter, T. Rosenkranz, A. N. Escalante-B., and A. Maier, “DeepFilterNet: Perceptually motivated real-time speech enhancement,” *arXiv preprint arXiv:2305.08227*, 2023, doi: 10.48550/arXiv.2305.08227.
- [10] J.-M. Valin, “A hybrid DSP/deep learning approach to real-time full-band speech enhancement,” *arXiv preprint arXiv:1709.08243*, 2017, doi: 10.48550/arXiv.1709.08243.
- [11] H. Zhao, S. Zarar, I. Tashev, and C.-H. Lee, “Convolutional-recurrent neural networks for speech enhancement,” *arXiv preprint arXiv:1805.00579*, 2018, doi: 10.48550/arXiv.1805.00579.
- [12] N. L. Westhausen and B. T. Meyer, “Dual-signal transformation LSTM network for real-time noise suppression,” in *Proc. Interspeech*, 2020, pp. 2477–2481, doi: 10.21437/Interspeech.2020-2631.
- [13] Y. Ephraim and D. Malah, “Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 32, no. 6, pp. 1109–1121, 1984, doi: 10.1109/TASSP.1984.1164453.
- [14] Y. Xia, S. Braun, C. K. A. Reddy, H. Dubey, R. Cutler, and I. Tashev, “Weighted speech distortion losses for neural-network-based real-time speech enhancement,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2020, pp. 871–875, doi: 10.1109/ICASSP40776.2020.9053523.
- [15] D. Takeuchi, K. Yatabe, Y. Koizumi, Y. Oikawa, and N. Harada, “Real-time speech enhancement using equilibrated RNN,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2020, pp. 851–855, doi: 10.1109/ICASSP40776.2020.9054597.
- [16] Abdulvahap, “Music Instrument Sounds for Classification,” *Kaggle.com*, 2024. [Online]. Available: <https://www.kaggle.com/datasets/abdulvahap/music-instrument-sounds-for-classification>
- [17] C. Valentini-Botinhao, “Noisy speech database for training speech enhancement algorithms and TTS models,” Univ. Edinburgh, 2017, doi: 10.7488/ds/2117.
- [18] C. Gorgolewski, “UrbanSound8K,” *Kaggle.com*, 2014. [Online]. Available: <https://www.kaggle.com/datasets/chrisfilo/urbansound8k>
- [19] M. S. Thu, “Audio Noise Dataset,” *Kaggle.com*, 2023. [Online]. Available: <https://www.kaggle.com/datasets/minisithu/audio-noise-dataset> [Accessed: Apr. 16, 2026].
- [20] H. Schröter, A. N. Escalante-B., T. Rosenkranz, and A. Maier, “DeepFilterNet2: Towards real-time speech enhancement on embedded devices for full-band audio,” *arXiv preprint arXiv:2205.05474*, 2022.